Recursive Orchestration at Scale: Architecting the Collider Framework with Pydantic-Graph

1. Introduction: The Evolution of Algorithmic Orchestration

The computational landscape for complex application orchestration is undergoing a fundamental paradigm shift. Historically, workflow management systems (WFS) have been dominated by the Directed Acyclic Graph (DAG) model, exemplified by tools like Apache Airflow. While robust for linear data engineering pipelines where tasks flow unidirectionally from ingestion to storage, the DAG model is increasingly insufficient for the emerging class of agentic and recursive applications. The "Collider" application, characterized by its recursive "Container/Link/Definition" architecture, represents a class of software that demands a more sophisticated theoretical foundation: the Finite State Machine (FSM) executed over a dynamic, potentially cyclic graph topology.

This report provides an exhaustive architectural analysis of integrating `pydantic-graph`—a library designed to bridge the gap between static typing and dynamic graph execution—into the Collider ecosystem. Unlike traditional orchestrators that treat graphs as static artifacts defined at deployment time, the Collider architecture requires "Just-In-Time" (JIT) graph compilation, where a "Container" is not merely a grouping mechanism but an executable graph entity that can itself function as an atomic "Definition" within a higher-order Container. This recursive capability, often described as "fractal orchestration," presents unique challenges in state isolation, dependency injection, and durable execution.

Our research synthesizes the current capabilities of the `pydantic-graph` ecosystem, analyzing both its stable class-based API and the emerging Beta functional API. We investigate the mechanisms for dynamic schema generation using Pydantic's `create_model` factories, the implementation of sophisticated control flow via "Decision" nodes, and the architecture of distributed state persistence required to support long-running, recursive processes. The evidence suggests that while `pydantic-graph` provides the necessary primitives—specifically through its Beta `GraphBuilder` pattern—successful integration requires a rigorous architectural bridging layer to translate the Collider's recursive data structures into executable Pydantic Graph definitions.[1]

1.1 The Collider Architectural Abstraction

To evaluate the fitness of `pydantic-graph`, we must first rigorize the definitions of the Collider's primary abstractions and map them to their computational counterparts within the Pydantic ecosystem. The Collider operates on a triad of concepts:

1. **Definition:** The atomic unit of logic or schema. In a static context, this might be a function or a class. In the Collider context, a Definition is a portable specification of work—an instruction set that validates inputs and produces outputs. This maps conceptually to the `BaseNode` in the stable Pydantic Graph API or a `Step` in the Beta API.[4]
2. **Link:** The connective tissue defining data flow and dependency. A Link is not merely a pointer; it acts as a "Router" or "Edge" that may contain conditional logic (e.g., "If validation fails, route to ErrorDefinition"). This maps to `Edge` objects and `Decision` nodes in `pydantic-graph`.[1]
3. **Container:** The bounding context that encapsulates a set of Definitions and Links. The critical architectural requirement of the Collider is **recursion**: a Container must be able to expose itself as a Definition. This implies that a Graph must be capable of execution as a Node within another Graph. This "Graph-as-Node" pattern is the central engineering challenge addressed in this report.[3]

1.2 Theoretical Gaps in Standard DAG Tools

The necessity for `pydantic-graph` arises from the limitations of standard DAG tools when applied to the Collider's requirements:

Architectural Requirement	Standard DAG Tool (e.g., Airflow)	Pydantic-Graph (FSM Approach)	Collider Implication
Topology Definition	Static, Deploy-time compilation	Dynamic, Runtime construction	Essential for loading Containers from DB configs.
Cyclic Execution	Strictly Forbidden (Acyclic)	Native Support (Cycles allowed)	Required for Agentic "Retry/Refine" loops.

State Management	External (XComs), loosely typed	Intrinsic, Strongly Typed (Generics)	Critical for passing complex context recursively.
Recursion	SubDAGs (deprecated/clunky)	Composable via Asyncio	Enables infinite nesting of Containers.

The following sections detail the implementation strategy, beginning with a deep technical analysis of the pydantic-graph core mechanics.

---

2. Core Mechanics: The Duality of Pydantic-Graph

As of early 2026, pydantic-graph exists in a transitional state, offering two distinct APIs: the Stable (Class-based) API and the Beta (Functional/Builder) API. Understanding the internal mechanics of these two modes is prerequisite to architecting the Collider, as different layers of the application will likely leverage different APIs.

2.1 The Stable API: Type-System Enforced Topology

The stable API represents the graph structure through Python's class inheritance and type hinting system. This approach creates a rigorous, "compile-time" (or static analysis time) contract for the graph's behavior.

2.1.1 Node Semantics and Return Types

In the stable API, a node is a class inheriting from BaseNode. The topology—which nodes connect to which—is defined exclusively by the return type annotation of the run method.[1]

```python
from pydantic_graph import BaseNode, GraphRunContext, End
from dataclasses import dataclass

@dataclass
class ColliderState:
    payload: dict

@dataclass
class DefinitionNode(BaseNode):
    async def run(self, ctx: GraphRunContext) -> 'NextDefinition | End':
        # Logic implementation
```

```
    return NextDefinition()
```

This design enforces that a transition from DefinitionNode to NextDefinition is valid at the code level. If a developer attempts to return a node type not listed in the union of the return annotation, static analysis tools (like MyPy) will flag it as an error.

Relevance to Collider: This strictness is highly advantageous for the **Definition** layer of the Collider. When a user defines an atomic operation (e.g., "Extract Email Entity"), the inputs and next steps for that specific logic are often fixed. Implementing Definitions as BaseNode subclasses provides a robust foundation for the "leaves" of the recursive tree.

2.1.2 State Injection and Generics

The stable API utilizes Python generics (Generic) to propagate state. The GraphRunContext acts as the vessel for this state. Because the state is strongly typed, the IDE and runtime can ensure that a Definition node only accesses valid attributes of the Collider's execution context.[4]

2.2 The Beta API: The GraphBuilder Pattern

The Beta API introduces a paradigm shift towards a "Builder" pattern, decoupling the node's logic from the graph's topology. This is the critical enabler for the **Container** layer of the Collider.[2]

2.2.1 Decoupling Logic from Topology

In the Beta API, nodes are defined as Step functions (async functions decorated with @builder.step). The connections between them are not defined in the function signature but are added procedurally via builder.add_edge().

```Python
from pydantic_graph.beta import GraphBuilder, StepContext

# Logic defined in isolation
async def generic_step(ctx: StepContext) -> str:
    return "result"

# Topology defined at runtime
def build_container_topology(config: dict):
    builder = GraphBuilder(state_type=ColliderState,...)
    # Dynamically wiring the graph based on configuration
    builder.add(builder.edge_from(builder.start_node).to(generic_step))
    return builder.build()
```

Relevance to Collider: The Collider's "Container" is essentially a configuration file (JSON/YAML) that describes which Definitions are linked. The Beta API functions as a **Compiler Target**. The Collider application parses the Container configuration and uses the `GraphBuilder` to instantiate an executable graph object in memory. This dynamic construction capability is missing from the stable API and standard DAG tools.

2.3 Internal Execution Model

Under the hood, both APIs rely on an asynchronous loop that iterates through the graph. The `GraphRun` object[7] maintains the cursor (current node) and the state. Crucially, the execution is driven by `asyncio`, which means that "blocking" a node to run a sub-graph (recursion) is a non-blocking operation for the underlying OS thread, allowing for high concurrency even with deep recursion depths.

---

3. The Definition Layer: Dynamic Schema Engineering

In the Collider architecture, a "Definition" is not just a block of code; it is a contract. It specifies the shape of data it accepts and the shape it produces. Since Definitions can be created or modified by users, the Python code cannot strictly hardcode these schemas as static classes. We must leverage Pydantic's dynamic model creation capabilities to bridge this gap.

3.1 Just-In-Time Schema Synthesis

Pydantic's `create_model` function is the engine for this dynamic typing. It allows the runtime creation of `BaseModel` subclasses that are indistinguishable from statically defined models to the validation engine.[8]

For the Collider, every Definition configuration must include an `input_schema` specification. The application must synthesize this into a Pydantic model before the graph executes.

```python
from pydantic import create_model, Field
from typing import Any, Dict

def synthesize_definition_schema(def_name: str, schema_config: Dict[str, Any]) -> type:
    """
    Converts a JSON-based schema definition into a Pydantic Model class.
    """
    fields = {}
    for field_name, attributes in schema_config.items():
        # Map string type names to Python types
        py_type = _resolve_type(attributes['type'])
        fields[field_name] = (py_type, Field(description=attributes.get('description')))
```

```
    return create_model(f'{def_name}Input', **fields)
```

3.2 Handling Merged and Overlapping Schemas

A significant challenge in the Link layer is when multiple upstream Definitions feed into a single downstream Definition. The downstream Definition effectively receives a union or merger of the upstream outputs.

Research indicates that `create_model` can handle multiple base classes, effectively merging fields from multiple models.[10] However, conflicting field types (e.g., `field_a` is `int` in Model A and `str` in Model B) will raise `RuntimeError` or `TypeError` during model creation unless explicitly handled.[12]

Strategy for Collider:

The Collider must implement a Schema Resolution Strategy within the Link layer.

1. **Union Strategy:** The input to the downstream Definition is typed as `Union`. This forces the logic to handle either case.[10]
2. **Intersection/Merge Strategy:** A new model is created that combines fields. If conflicts exist, the Collider compiler must throw a "Linkage Error" before execution begins, informing the user that the Link is invalid due to type mismatch.

3.3 Integrating Dynamic Models with Graph Nodes

In the Beta API, a `Step` function receives a `StepContext`. While the `StepContext` is generic, the runtime data inside `ctx.inputs` is dynamically typed.

Python

```python
@builder.step
async def execute_definition(ctx: StepContext) -> Any:
    # 1. Retrieve the synthesized schema for this specific node
    node_id = ctx.node_id
    schema_model = ctx.deps.schema_registry.get_input_model(node_id)

    # 2. Perform Runtime Validation
    # Pydantic-graph ensures strict transitions, but the data payload
    # must be validated against the user-defined schema.
    validated_data = schema_model.model_validate(ctx.inputs)

    # 3. Execution
```

```
    return await run_user_logic(validated_data)
```

This pattern ensures that the rigorous data validation Pydantic is known for is applied even within a highly dynamic, user-configured graph topology.[8]

---

4. The Link Layer: Advanced Control Flow and Routing

The "Link" in the Collider architecture is responsible for directing the flow of execution. In a simple DAG, a link is a directed edge. In an FSM, a link is a transition that may depend on the state of the system. `pydantic-graph` enables this through **Decision Nodes** and conditional edges.

4.1 Decision Nodes as Routers

The Beta API introduces specific support for `Decision` nodes (or Router steps) that do not perform work but rather evaluate the state to determine the next step.[2]

Architectural Mapping:

A "Link" in Collider that contains logic (e.g., "If score > 50") maps directly to a Decision node in pydantic-graph.

A "Link" that is unconditional maps directly to an Edge added via builder.add_edge().

4.2 Implementation of Conditional Logic

The `GraphBuilder` allows creating complex branching logic using the `.match()` method on edges originating from a decision node.

Python

```
@builder.step
async def route_based_on_outcome(ctx: StepContext) -> str:
    # Logic derived from the Link configuration
    if ctx.state.last_result['status'] == 'success':
        return 'success_path'
    return 'failure_path'

# Wiring the graph
builder.add(
    builder.edge_from(route_based_on_outcome).to(next_def).match('success_path'),
    builder.edge_from(route_based_on_outcome).to(error_handler).match('failure_path')
)
```

This establishes a clear translation layer: **Collider Link Logic -> Python Router Function**.

4.3 Cycle Detection and Topological Safety

Since the Collider allows users to define Links, they can inadvertently (or intentionally) create infinite loops. While pydantic-graph allows cycles (it is an FSM), "structural cycles" (A -> B -> A) without exit conditions can lead to infinite execution costs.

Before compiling the graph, the Collider engine must perform a **Topological Analysis**.[14]

1. Convert the Collider definitions into a NetworkX DiGraph.[16]
2. Use networkx.simple_cycles to detect loops.
3. **Policy Enforcement:** If a cycle is detected, the engine must verify that the loop contains at least one "State Mutating Decision Node" that acts as an exit gate (e.g., a counter increment or a convergence check). If the loop is purely structural (infinite loop), the graph build should be rejected.[18]

---

5. The Container Layer: Recursive Execution Architecture

The "Container" is the most complex abstraction in the Collider. It is a Graph that acts as a Node. This recursive capability is not natively supported by a single primitive in pydantic-graph (e.g., there is no GraphNode class). Consequently, the core contribution of this research is the definition of the **Recursive Graph Node (RGN)** pattern.

5.1 The Recursive Graph Node Pattern

To implement recursion, we define a specialized Node (in Stable API) or Step (in Beta API) whose sole responsibility is to instantiate and execute a *sub-graph*.

5.1.1 Implementation Detail

The ContainerNode must perform three critical operations:

1. **JIT Compilation:** Retrieve the configuration for the target Container and build the pydantic-graph object in memory.
2. **State Bridging:** Transform the generic GlobalState of the parent into the specific InitialState required by the child Container.
3. **Async Execution:** Await the completion of the child graph and map the results back.

Python

```python
@dataclass
class ContainerNode(BaseNode):
    container_config_id: str
```

```
async def run(self, ctx: GraphRunContext) -> 'NextNode':
    # 1. Fetch Definition
    config = ctx.deps.registry.get_container(self.container_config_id)

    # 2. Build Sub-Graph (Beta Builder)
    sub_builder = GraphBuilder(state_type=ChildState,...)
    #... logic to populate sub_builder from config...
    sub_graph = sub_builder.build()

    # 3. State Bridging (The Airlock)
    # Deep copy essential context, isolate working memory
    child_state = ChildState(
        trace_id=ctx.state.trace_id,
        input_payload=ctx.state.current_payload
    )

    # 4. Recursive Execution
    # This await suspends the parent node until the child completes
    result = await sub_graph.run(child_state, deps=ctx.deps)

    # 5. Output Integration
    ctx.state.current_payload = result.output

    return NextNode()
```

5.2 State Scoping and Isolation

Recursion introduces the "Scope" problem. Does the inner Container share the outer Container's state?

- **Shared State:** Risk of "pollution" where the inner container modifies variables the outer container relies on.
- **Isolated State:** Safer, but requires explicit mapping of inputs and outputs.

Recommendation: The Collider should strictly enforce **Isolated State**. The `ContainerNode` acts as an "Airlock." It explicitly marshals data *in* (via the `ChildState` constructor) and marshals data *out* (via the `RunResult` output). This prevents side effects and makes debugging recursive graphs significantly easier.[5]

5.3 Stack Depth and Memory Considerations

Since pydantic-graph runs on asyncio, "recursion" here is actually "nested awaiting." Python's asyncio loop is resilient to deep nesting compared to synchronous recursion (which hits stack

limits). However, each level of recursion consumes memory for the GraphRun instance and its state.

Mitigation: The ContainerNode should implement rigorous error handling. If the child graph raises an exception, the ContainerNode must catch it, wrap it in a ContainerExecutionError including the container_id, and propagate it to the parent for handling (or routing to a failure link).7

---

6. Parallelism and Concurrency: Fork-Join Semantics

Collider Definitions often need to run in parallel—for example, a "Research Container" that scrapes five different URLs simultaneously. The Beta API provides the **Fork-Join** mechanism to support this.

6.1 The Mechanics of Forking

The Beta API supports two types of parallel "forks" [5]:

1. **Broadcast:** Sending the same data to multiple distinct steps. (e.g., Run SentimentAnalysis and KeywordExtraction on the same text).
2. **Map (Spread):** Iterating over a list and running the *same* step for each item. (e.g., Run ScrapeUrl for each URL in state.url_list).

Integration: The GraphBuilder exposes .map() and .to(step_a, step_b) methods. The Collider compiler must translate "Parallel Links" in the configuration into these builder calls.

6.2 The Join and Reducer Pattern

A critical rule in pydantic-graph is that forks *must* be joined before the graph can converge. This is handled by **Join Nodes** equipped with **Reducers**.[21]

The Reducer's Role:

When multiple parallel branches complete, they produce multiple results. The Reducer function determines how to merge these into the single state object of the graph.

- *List Append Reducer:* reduce_list_append collects all outputs into a list.
- *Custom Reducer:* The Collider might require a "Merge Strategy" (e.g., dictionary update, or even using an LLM to synthesize parallel text outputs).

```python
from pydantic_graph.beta.join import reduce_list_append
```

```
# Define the join node
collector = builder.join(reduce_list_append, initial_factory=list)

# Wiring the parallel map
builder.add(
    builder.edge_from(start_node).map(lambda s: s.items).to(worker_step),
    builder.edge_from(worker_step).to(collector)
)
```

Insight: In a recursive environment, if a ContainerNode runs internally in parallel (e.g., multiple instances of the same Container running on different data), the ContainerNode itself acts as the "Worker Step" in the map operation.

---

7. Durable Execution and Distributed State Persistence

For "Deep Research" scenarios where a Collider graph might run for hours or days, executing purely in memory is a liability. If the server restarts, progress is lost. The system requires **Durable Execution**.

7.1 The Persistence Interface

pydantic-graph provides a BaseStatePersistence protocol.[7] Implementing this allows the graph to snapshot its state after every step.

Snapshot Data Model:

A snapshot typically contains:

- graph_run_id: The unique ID of the execution.
- node_id: The last successfully completed node.
- state: The serialized state object.
- timestamp: Execution time.

7.2 Recursive Persistence Strategy

The standard persistence mechanism saves the state of the *current* graph run. It does *not* automatically save the state of a nested graph run initiated inside a node (the ContainerNode).

The Fractal Persistence Problem: If the Parent Graph is persisted, but the Child Graph running inside ContainerNode is not, a restart will cause the ContainerNode to restart the Child Graph from the beginning.

Solution: The ContainerNode must implement Compound Persistence IDs.

1. The Parent Graph has `RunID: A`.
2. The `ContainerNode` derives a Child Run ID: `RunID: A/Step:ContainerNode`.
3. The `ContainerNode` passes this derived ID to the Child Graph's persistence context.
4. This creates a hierarchical tree of persistence keys in the database (e.g., Redis).
5. On resume, the `ContainerNode` checks if `RunID: A/Step:ContainerNode` exists in the DB and resumes the child graph instead of restarting it.

7.3 Integration with Temporal / DBOS

For enterprise-grade durability, Pydantic AI supports integration with Temporal and DBOS.[24] These platforms provide the orchestration layer that guarantees code execution. In this setup, `pydantic-graph` defines the *logic* of the state machine, while Temporal handles the *retries, timeouts, and distributed execution* of the activities (Definitions). This is the robust path for production deployments.

---

8. Observability and Visualization

Understanding the behavior of a recursive, dynamic graph is impossible without advanced visualization.

8.1 Mermaid Diagram Generation

pydantic-graph natively supports generating Mermaid JS diagrams via graph.mermaid_code().4

Customization: The architecture should utilize docstring_notes and get_note methods on Nodes to inject runtime debug info into the diagrams (e.g., showing the specific schema name used by a Definition node).28

Recursive Visualization:

Since the full graph is not known at compile time (due to JIT container loading), we cannot generate one massive "Master Diagram."

Strategy: The user interface for Collider should support Drill-Down Visualization.

1. Render the Top-Level Graph.
2. Render `ContainerNodes` as distinct sub-graph shapes.
3. On click, fetch the configuration for that Container, compile it, generate its specific Mermaid code, and display it in a modal or nested view.

8.2 Distributed Tracing (OpenTelemetry)

pydantic-graph includes auto_instrument capabilities.7 This should be enabled to generate OpenTelemetry spans for every node execution.

Trace Propagation: The traceparent header (W3C standard) must be passed from the Parent State into the Child State within the ContainerNode. This ensures that a trace viewer (like Jaeger) renders the recursive execution as a single continuous trace, rather than disjointed traces.

---

9. Conclusion: The Unified Architecture

The integration of pydantic-graph into the Collider application offers a robust solution to the challenge of recursive orchestration. By aligning the **Stable API** with the rigid requirements of atomic **Definitions** and the **Beta API** with the dynamic topology of **Containers**, we establish a framework that is both type-safe and highly flexible.

The proposed **Recursive Graph Node (RGN)** pattern effectively bridges the gap between the two, enabling the "fractal" architecture where Containers can be infinitely nested. When augmented with **Dynamic Schema Synthesis** for validation, **Decision Nodes** for intelligent routing, and **Hierarchical State Persistence** for durability, pydantic-graph transforms from a simple library into a comprehensive engine capable of powering the next generation of agentic applications.

Summary of Recommendations

Layer	Recommended Pydantic-Graph Primitive	Implementation Pattern
Definition	**Stable API (BaseNode)**	Use create_model for dynamic input validation.
Link	**Beta API (Decision/Edge)**	Compile logic links to Router steps; simple links to Edges.
Container	**Beta API (GraphBuilder)**	Use JIT compilation to build graphs from config at runtime.
Recursion	**Custom RGN Pattern**	Wrap Child Graph execution in a Parent Node with state bridging.

Persistence	**Redis / Temporal**	Implement compound keys (`ParentID/ChildID`) to persist nested states.

This architecture delivers on the original promise of the Collider: a system where the "structure" of the computation is as programmable and composable as the code itself.